TinyGPT: Training Pipeline, Quantitative Analysis, Evaluation, and Deployment

Technical Report | Version 1.0

26 October 2026

Contents

1 Purpose and scope **2**
2 Model architecture **2**
3 Training pipeline and experiment history **3**
4 Pretraining results **7**
5 Mathematical analysis **10**
6 Final SFT results and distribution shift **11**
7 Generation benchmark and qualitative findings **12**
8 GGUF conversion and local deployment **13**
9 Limitations and threats to validity **14**
10 Conclusion **15**
11 Artifacts and reproducibility **15**
References **15**

Evidence base: Supplied notebooks, saved notebook outputs, consolidated Stage 3/Stage 4 logs, and the recorded benchmark report.

Scope and evidence standard. The report distinguishes recorded results from derived calculations and unresolved gaps. Dataset-license compatibility and model redistribution permissions remain subject to a separate review before public release.

Abstract

The primary objective of TinyGPT was to build a capable conversational AI model from scratch. That objective was not achieved: the final 51,082,752-parameter models remain unreliable and display weaknesses in arithmetic, coding, instruction-following, factual grounding, and hallucination resistance. The project did, however, successfully execute and document a complete small-language-model pipeline: model initialization, four pretraining stages, supervised fine-tuning (SFT), comparative evaluation, and FP16 GGUF export for local inference. The model uses a GPT-2-style decoder-only Transformer, the GPT-2 tokenizer (vocabulary size 50,257), and a 256-token context length. The four pretraining stages processed 3,050,037,248 input-token positions in total, approximately 59.71 processed positions per parameter. Stage 4 recorded validation cross-entropy loss of 3.6472 under its pretraining evaluation procedure. After SFT, Mix A obtained the lower chat validation loss (2.6181), while Mix B obtained the lower Q&A validation loss (3.1494). A 27-prompt benchmark and manual LM Studio testing exposed significant generation failures. The result is therefore not a reliable general-purpose assistant; it is a documented end-to-end experiment whose results show that pipeline completion and lower task-specific validation loss do not guarantee strong conversational behaviour.

1 Purpose and scope

The primary goal was to build a good conversational AI model from scratch. The final model did not meet that goal: testing showed unreliable answers, hallucinations, repetition, and weak arithmetic, coding, reasoning, and instruction-following. This shortfall is an important outcome and is stated plainly rather than hidden behind the fact that the pipeline ran successfully.

The project nevertheless completed the planned engineering workflow for a small autoregressive model: architecture configuration, pretraining on web-text data, continued pretraining, dialogue/instruction SFT, comparison of two SFT mixtures, held-out loss evaluation, prompt-based generation testing, and conversion to GGUF for local inference. This made it possible to examine the relationships and gaps between training metrics and actual generated behaviour. The report focuses on what the saved notebooks and logs establish, including the limits of the evidence.

1.1 What “from scratch” means here

The model weights were initialized from a GPT-2-compatible configuration rather than loaded from official pretrained GPT-2 weights. The implementation uses the Hugging Face GPT2Config / GPT2LMHeadModel implementation, and the tokenizer is the existing GPT-2 tokenizer. Thus, the weights were trained from scratch, while the architecture implementation and tokenizer were reused; neither a new Transformer implementation nor a new tokenizer was built from scratch.

1.2 Project questions

- Can a compact 51M-parameter GPT-style model be trained through an iterative pretraining pipeline on approximately 3.05B processed token positions?
- How does continued pretraining change the recorded held-out language-modelling loss?
- How do two SFT mixtures—chat-heavy Mix A and Q&A-heavy Mix B—compare on held-out chat and Q&A response tokens?
- How well do the final variants behave on a small prompt-based generation benchmark, and what limitations remain?

2 Model architecture

TinyGPT is an autoregressive, decoder-only Transformer language model. It predicts a distribution for the next token conditioned on the preceding token sequence, using masked self-attention and feed-forward blocks. This general architecture follows the Transformer and GPT-family language-modelling approach [1, 2].

Specification	Recorded value
Architecture implementation	GPT-2-style causal language model (`GPT2LMHeadModel`)
Total parameters	51,082,752
Transformer blocks	8

Specification	Recorded value
Attention heads per block	8
Hidden / embedding dimension	512
Context length	256 tokens
Tokenizer	GPT-2 tokenizer
Vocabulary size	50,257
EOS token ID	50,256
Padding token for training/inference	EOS token ID (50,256)
Final pretraining objective	Causal next-token prediction
Final deployment format	FP16 GGUF (not quantized)

The parameter count was verified by summing the model's trainable parameters in the supplied notebooks. The following approximate component breakdown is consistent with the 8-layer GPT-2 configuration and tied token-embedding/output weights:

Component	Calculation	Parameters
Token embedding	$50{,}257 \times 512$	25,731,584
Positional embedding	256×512	131,072
Eight Transformer blocks	$8 \times 3{,}152{,}384$	25,219,072
Final layer normalization	512 scale + 512 bias	1,024
Total	**Sum of components**	**51,082,752**

For one Transformer block with model width $d_{model} = 512$ and feed-forward width $d_{ff} = 2048$, the parameter sum is:

- Attention QKV projection: $512 \times 1536 + 1536 = 787{,}968$; attention output projection: $512 \times 512 + 512 = 262{,}656$; attention subtotal: 1,050,624.
- MLP input projection: $512 \times 2048 + 2048 = 1{,}050{,}624$; MLP output projection: $2048 \times 512 + 512 = 1{,}049{,}088$; MLP subtotal: 2,099,712.
- Two LayerNorms: $2 \times (512 + 512) = 2{,}048$.

Per-block total: $1{,}050{,}624 + 2{,}099{,}712 + 2{,}048 = 3{,}152{,}384$ parameters.

The output language-model head shares its weights with the token embedding table in the standard GPT-2 configuration, so a second independent vocabulary-by-hidden-size matrix is not added to this count.

3 Training pipeline and experiment history

There were four pretraining stages. SFT experiments were conducted after early stages, after Stage 3, and after Stage 4. The final release candidates are the Stage 4 SFT Mix A and Mix B checkpoints. The earlier SFT experiments are historical experiments and are not inputs to the final Stage 4 checkpoint lineage.

3.1 Main production lineage

- **Stage 1 — initial pretraining:** random-initialized 51M model trained on FineWeb, approximately 50M token positions.
- **Stage 2 — continued pretraining:** Stage 1 checkpoint continued on FineWeb-Edu, approximately another 50M positions. The corrected run used `pretrain2_best.pt` as the next stage's input.
- **Stage 3 — continued pretraining:** approximately 950M additional FineWeb-Edu positions, starting from `pretrain2_best.pt`. The full Stage 3 run was split across sessions because of runtime limits; the continuation reached 950,009,856 positions.
- **Stage 4 — continued pretraining:** approximately 2B additional FineWeb-Edu positions, started from the full Stage 3 `real_latest.pt` checkpoint. The Stage 4 token counter and data stream were reset for the new stage, while the notebook's continuation code preserved the

AdamW and AMP scaler states. The run reached 2,000,027,648 Stage 4 positions and produced `stage4_best.pt`.

- **Stage 4 SFT data preparation:** prepared train/dev records and formed the final chat-heavy and Q&A-heavy mixtures.
- **Final SFT:** independently fine-tuned two copies of `stage4_best.pt` to produce Mix A and Mix B checkpoints.
- **Evaluation and deployment:** tested Mix A and Mix B using the automated prompt benchmark and exported both to FP16 GGUF for LM Studio.

The earlier Stage 1/Stage 2 SFT experiments and Stage 3 SFT experiment are retained as experimental history. Stage 4 did not start from a Stage 3 SFT checkpoint; it continued from the Stage 3 pretraining checkpoint.

Checkpoint lineage and optimizer state: Stage 3 loaded model weights from `pretrain2_best.pt` and initialized a fresh AdamW optimizer with a new warmup/learning-rate schedule; it did not resume Stage 2's optimizer moments. Stage 4, by contrast, loaded Stage 3's full `real_latest.pt` checkpoint and restored its AdamW optimizer and AMP GradScaler states, while starting a fresh Stage 4 token counter, data-stream cursor, and learning-rate schedule. The Stage 2 log records `pretrain2_best.pt` and `pretrain2_latest.pt` both saved at Stage 2 step 6,104 after the final validation loss of 5.0101; exact byte-for-byte identity of the two files was not checked separately.

3.2 Pretraining stages and settings

Setting	Stage 1	Stage 2 (corrected run)	Stage 3	Stage 4
Dataset	`HuggingFaceFW/fineweb`	`HuggingFaceFW/fineweb-edu`, config `sample-10BT`	Same FineWeb-Edu config	Same FineWeb-Edu config and pinned revision inherited from Stage 3
Starting weights	Random initialization	Stage 1 `latest.pt`	`pretrain2_best.pt`	Stage 3 `real_latest.pt`
Target positions	50M	50M	950M	2,000M
Recorded processed positions	≈49,999,872	49,999,872	950,009,856	2,000,027,648
Context	256	256	256	256
Batch / accumulation	8 / 4	8 / 4	32 per GPU × 2 GPUs / 2	32 per GPU × 2 GPUs / 2
Input positions per optimizer step (nominal)	8,192	8,192	32,768	32,768
Peak learning rate	3×10^{-4}	3×10^{-4}	4×10^{-4}	2.5×10^{-4}
Learning-rate floor	Cosine schedule from notebook	3×10^{-5}	4×10^{-5}	2.5×10^{-5}
Optimizer	AdamW	AdamW	AdamW	AdamW
Weight decay	0.01	0.01	0.01	0.01
Gradient clipping	Not consistently reported in Stage 1 config	See notebook implementation	1.0	1.0
Dropout in model config	0.1	Inherited from Stage 1 checkpoint config	0.0	0.0

Setting	Stage 1	Stage 2 (corrected run)	Stage 3	Stage 4
Validation setup	FineWeb, target 1M tokens	Held-out FineWeb, target 250k tokens, corrected next-token objective	Deterministic validation cache, target 250k tokens; validate every 1,000 steps	Deterministic validation cache, target 250k tokens; validate every 1,000 steps
Environment	Google Colab	Google Colab, Tesla T4	Kaggle, 2 × Tesla T4 (14.56 GiB each)	Kaggle, 2 × Tesla T4 (14.56 GiB each)

Configuration caveat: Stage 1's dropout was 0.1. Stage 2 loads the saved model configuration. The Stage 3 and Stage 4 code explicitly constructs the compatible GPT-2 configuration with dropout set to 0.0. This is a training configuration change between stages, although the parameter count and main architecture dimensions remain the same.

For Stages 3 and 4, each optimizer step nominally processed 32,768 input-token positions: 32 examples per GPU × 2 GPUs × 256 positions × 2 gradient-accumulation steps. The Stage 2-to-Stage 3 transition is a weights-only handoff rather than a full optimizer resume; this differs from Stage 4, which restored Stage 3's optimizer and scaler states. Stage 3's run configuration capped linear warmup at 200 steps, then used its own cosine schedule. Stage 4 used a peak learning rate of 2.5×10^{-4}, a 2.5×10^{-5} floor, a maximum warmup of 150 steps, weight decay 0.01, and a gradient clipping norm of 1.0. Checkpoints were saved periodically, and validation was run every 1,000 steps.

3.3 SFT methodology

SFT examples use role-formatted text such as `User: ...` and `Assistant: ...`. Training labels mask the prompt tokens (label value `-100`) so that the loss is applied to assistant-response tokens rather than the entire prompt. The assistant response includes the end-of-sequence token. This trains the model to predict target assistant content conditioned on the input conversation.

3.3.1 Earlier SFT experiments

The first Stage 1 SFT run used DailyDialog, with 41,637 training examples and 3,851 validation examples, two epochs, batch size 8, gradient accumulation 4, and learning rate 5×10^{-5}. Recorded epoch metrics were:

Epoch	Training loss	Validation loss
1	3.7404	3.6354
2	3.5665	3.5891

A later Stage 2 SFT run also used DailyDialog for two epochs, with a learning-rate schedule from 1×10^{-4} toward 1×10^{-5}. The saved notebook output does not preserve a complete and reliable set of epoch-level validation losses for this second SFT run, so those missing metrics are not reconstructed here.

3.3.2 Stage 3 SFT experiment

The Stage 3 SFT experiment used DailyDialog, Everyday Conversations, Dolly 15k, and OpenAssistant data in two mixtures with approximately 1.2M active assistant tokens per mixture. It is retained as an exploratory branch. Its recorded final results were:

Evaluation	Stage 3 base	Stage 3 Mix A	Stage 3 Mix B
Chat validation loss	4.1225	3.0027	3.1022

Evaluation	Stage 3 base	Stage 3 Mix A	Stage 3 Mix B
Q&A validation loss	3.4912	3.3007	3.1911
Stage 3 SFT cache loss (labelled “FineWeb loss” by notebook)	3.6440	3.8822	3.8098

The Stage 3 base value of 3.6440 in this experiment differs from the Stage 3 pretraining final validation loss of 3.7688; the cause was not established and likely reflects a different validation cache or loss aggregation, so the two should not be compared directly.

These Stage 3 SFT values belong to that experiment’s evaluation and are not merged with the later Stage 4 SFT metrics.

3.3.3 Final Stage 4 SFT data

The saved preparation/training outputs record 83,666 training records and 10,046 development records. Training sources/counts recorded for the final preparation run were:

Source	Training records
SODA	39,668
EmpatheticDialogues	18,000
Dolly	8,766
OpenAssistant (OASST1)	1,950
DailyDialog	10,131
Everyday Conversations	4,201
Synthetic recall examples generated in the notebook	950
Total	**83,666**

A separate handwritten-pairs input file was not attached in the saved final data-preparation run; the 950 retained examples in the counts above are reported as `synthetic_recall`. The tokenizer and maximum context length were kept at GPT-2 vocabulary 50,257 and 256 tokens.

Final SFT setting	Recorded value
Base checkpoint	`stage4_best.pt`
Hardware	2 × Tesla T4 (14.6 GiB each)
Epochs	2 per mix
Per-GPU batch size	16
Gradient accumulation	2
Global batch per optimizer update	Exactly 64 sequences (16 sequences/GPU × 2 GPUs × 2 gradient-accumulation steps)
Peak learning rate	5×10^{-5}
Learning-rate floor	5×10^{-6}
Warmup	5% of scheduled steps
Weight decay	0.01
Gradient clipping norm	1.0
Dropout	0.0
Optimizer	Fresh AdamW optimizer for each mix

Mix A targets a 70% chat / 30% Q&A split by active assistant-token budget for the chat+Q&A portion. Mix B targets 30% chat / 70% Q&A. The same 950 preserved synthetic-recall records (13,913 active tokens) are included in both mixtures. The `build_mix()` / `feasible_remaining` logic avoids oversampling

and constrains each mixture by the available chat and Q&A pool capacities; the available short-Q&A pool constrained the larger 70%-Q&A mix, so Mix B used fewer examples and fewer active tokens. The executed notebook output records 403,051 active short-Q&A tokens in Mix B, plus the 13,913 preserved synthetic-recall tokens, for a total active-token budget of 589,738. Because selection happens in whole examples, the target percentage is approximate. The recorded total active-token budget differs between mixtures: Mix A has 1,200,046 active tokens, while Mix B has 589,738. Therefore, the comparison is not perfectly budget-matched; observed differences cannot be attributed solely to the chat/Q&A ratio.

4 Pretraining results

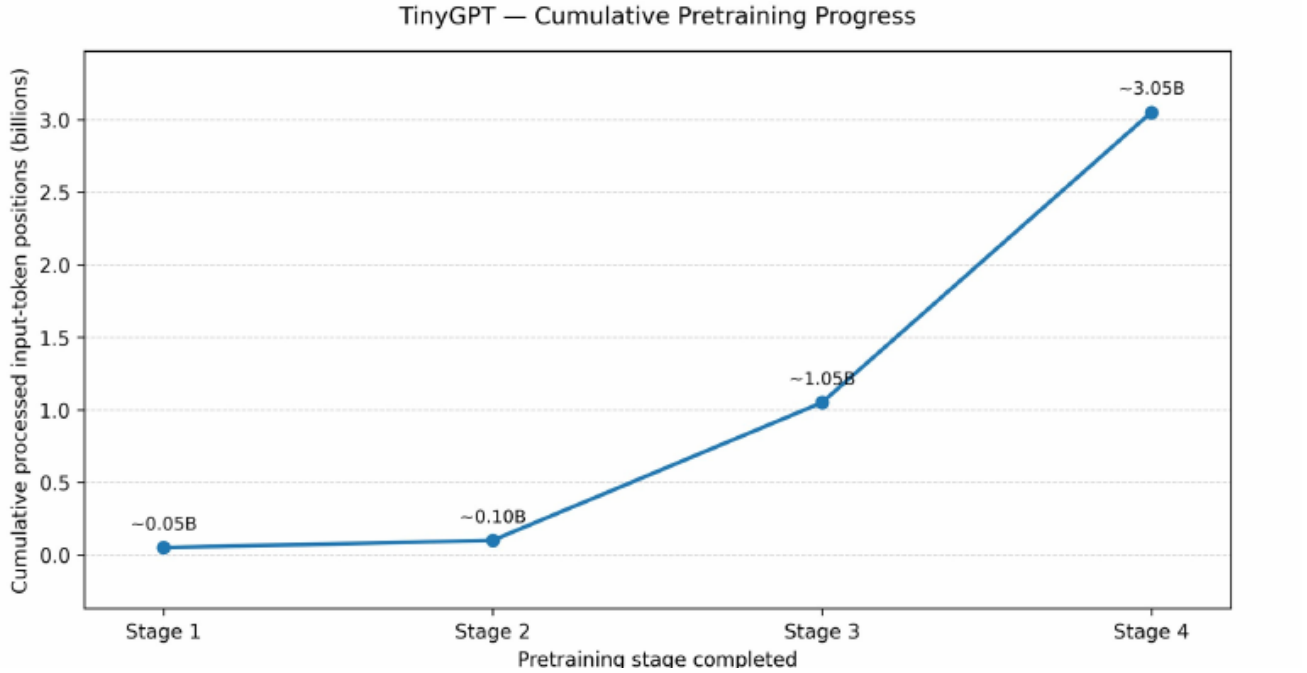
The pretraining runs produced the following recorded final validation values. These values are reported alongside the evaluation setup to avoid implying that measurements obtained from different code paths or caches are automatically identical.

Stage	Processed positions in stage	Cumulative positions	Starting/baseline validation loss	Final recorded validation loss
Stage 1	≈49,999,872	≈49,999,872	Not recorded (initial train loss ≈9.08)	6.4932 (FineWeb validation in Stage 1 notebook)
Stage 2	49,999,872	99,999,744	7.1034 (corrected baseline objective)	5.0101
Stage 3	950,009,856	≈1,050,009,600	4.7255 (Stage 3 validation-cache baseline)	3.7688
Stage 4	2,000,027,648	3,050,037,248	3.7688 (Stage 4 starting reference)	3.6472

Stage 1 and Stage 2 metrics should not be interpreted as a strict controlled loss curve because their validation token budgets and evaluation/data-construction paths differ. In the Stage 2 notebook, “corrected baseline objective” means re-evaluating the starting checkpoint with the Stage 2 causal language-modelling routine: the input token IDs are passed as labels, GPT-2 applies its standard one-token causal shift internally, an all-ones attention mask is supplied, and the held-out general FineWeb stream is evaluated with a 250,000-position target budget. Stage 1’s final validation number came from its own FineWeb validation loader with a different target budget (approximately one million positions). The saved artifacts do not establish a single exact cause for the higher Stage 2 baseline of 7.1034 versus Stage 1’s final 6.4932; this difference is therefore flagged as not directly comparable rather than attributed to model degradation.

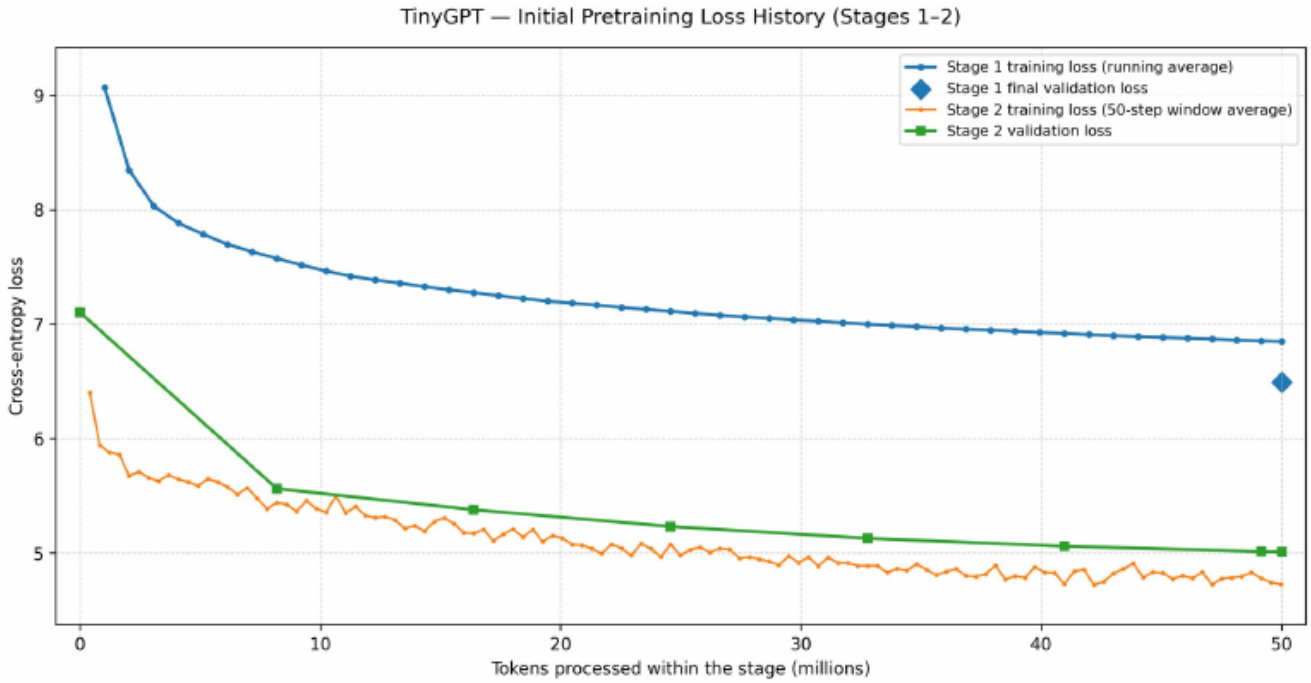
The Stage 2 final log reports both `pretrain2_best.pt` and `pretrain2_latest.pt` saved at step 6,104 after the final loss of 5.0101. Stage 3 evaluates that weights checkpoint on a newly built deterministic FineWeb-Edu validation cache, obtaining a baseline of 4.7255. This most likely explains why the Stage 2 final 5.0101 and Stage 3 starting 4.7255 differ: the former uses held-out general FineWeb, while the latter uses FineWeb-Edu (not tested separately). Stage 4 began from Stage 3’s continuation checkpoint and used a 3.7688 starting reference, ending at 3.6472.

Figure 1. Cumulative pretraining progress



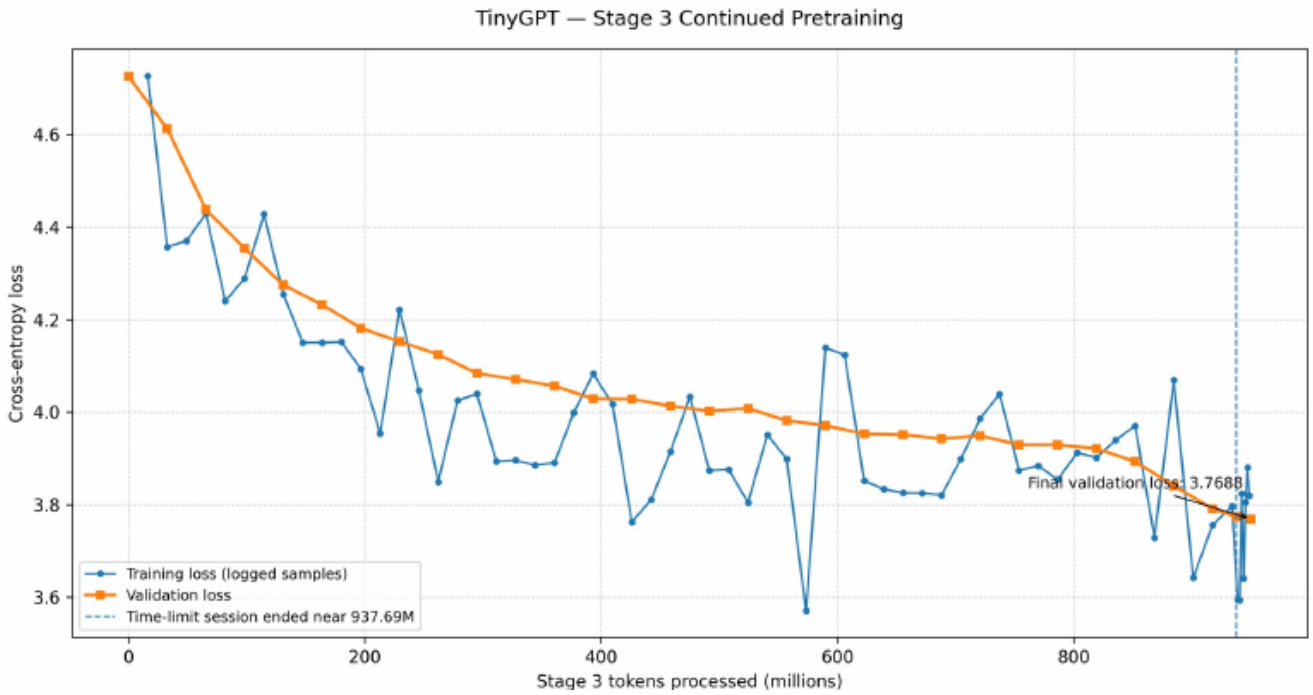
The recorded cumulative budget grows from ~50M positions after Stage 1 to ~3.05B positions after Stage 4. The counts refer to processed input-token positions, not a count of unique corpus tokens. Stage 1–2 values are approximate budgets; Stage 3–4 use recorded counters.

Figure 2. Initial pretraining loss history (Stages 1–2)



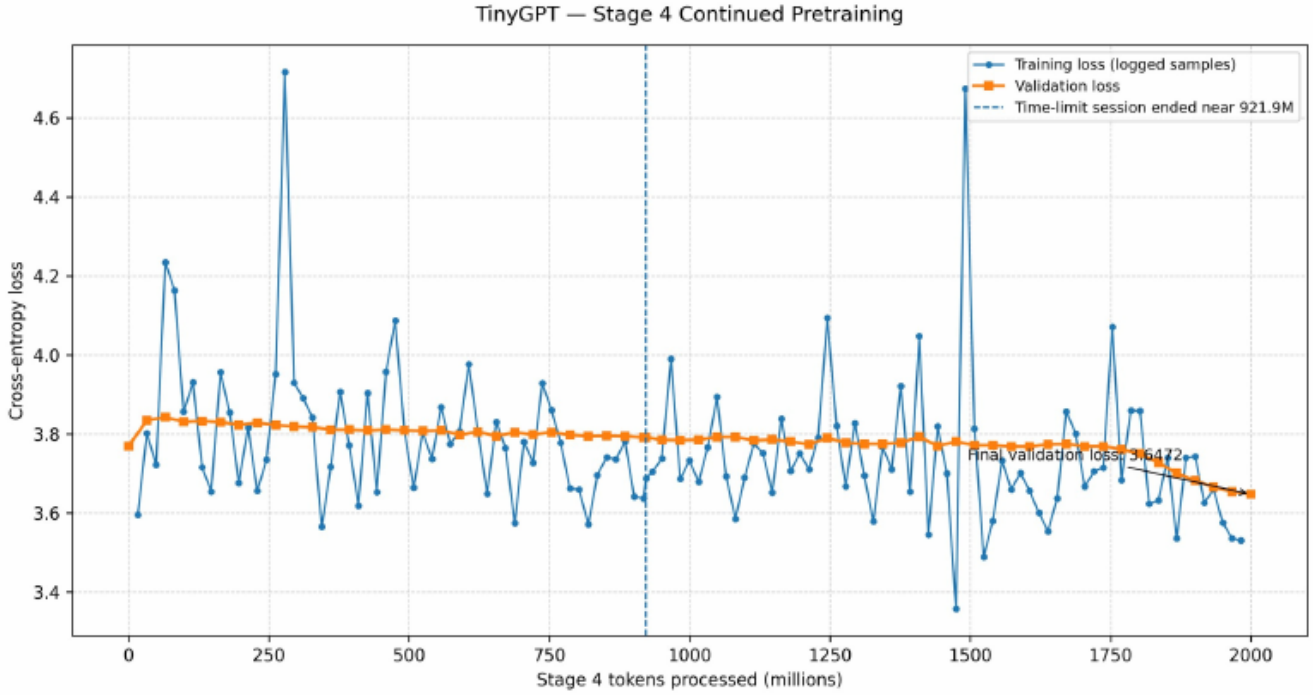
Stage 1's saved output includes the running-average training loss and final validation loss. Stage 2's corrected run includes periodic training-loss values and validation readings. The curves use the log aggregations produced by each respective notebook and should not be treated as perfectly comparable between stages.

Figure 3. Stage 3 continued pretraining



The Stage 3 run was interrupted by a runtime time limit at approximately 937.69M positions and then resumed. The final count reached 950,009,856 positions with validation loss 3.7688. The chart is based on 64 logged training-loss samples and 30 post-start validation readings, plus the recorded baseline. Samples are sparse log snapshots, not every optimizer step.

Figure 4. Stage 4 continued pretraining



Stage 4 was also run across sessions: its first session ended at approximately 921.895M Stage 4 positions, after which training resumed. The full Stage 4 run reached 2,000,027,648 positions and final validation loss 3.6472. The validation loss temporarily rose from the 3.7688 baseline to approximately 3.84 early in Stage 4 before declining. The chart is based on 122 logged training-loss samples and 62 validation readings, plus the Stage 3 baseline (3.7688). The early validation bump may partly reflect the fresh Stage 4 learning-rate schedule; causation is not established. One plausible contributor is the new Stage 4 schedule: Stage 3 ended at its 4.0×10^{-5} learning-rate floor, while Stage 4 used a fresh warmup toward 2.5×10^{-4} (6.25 times the Stage 3 floor). This is a plausible mechanism, not a demonstrated causal explanation; the run did not include a controlled experiment isolating the learning-rate restart.

5 Mathematical analysis

5.1 Cross-entropy loss

For a target sequence of tokens, next-token language modelling aims to maximise the probability assigned to each correct next token given the preceding context. The mean negative log-likelihood (cross-entropy) can be written as:

$$L = -\frac{1}{N} \sum_{t=1}^{N} \ln p_\theta(x_t \mid x_{<t})$$

where N is the number of scored target tokens, x_t is the correct token at position t, and p_θ is the model's predicted probability. A lower loss indicates that the model assigned higher probability, on average, to the correct target tokens in that evaluation set. It is not a percentage accuracy score and does not directly measure reasoning or factual reliability.

5.2 Perplexity

When the reported loss is average cross-entropy using natural logarithms, perplexity is:

$$\text{PPL} = e^L$$

For the Stage 3 final validation loss of 3.7688, the derived perplexity is approximately $e^{3.7688} = 43.33$. For the Stage 4 final validation loss of 3.6472, it is approximately $e^{3.6472} = 38.37$. These derived values are suitable for a within-pipeline comparison only to the extent that the validation token distribution, tokenizer, and loss calculation are consistent. Perplexity is not the percentage of questions answered correctly.

5.3 Cumulative processed-token budget

The recorded stage budgets are:

$$T_{\text{total}} = T_1 + T_2 + T_3 + T_4$$

$$\begin{aligned} T_{\text{total}} = {} & 49{,}999{,}872 + 49{,}999{,}872 \\ & + 950{,}009{,}856 + 2{,}000{,}027{,}648 \end{aligned}$$

$$\boxed{T_{\text{total}} = 3{,}050{,}037{,}248 \text{ processed input-token positions}}$$

This is the cumulative processed-position budget recorded across stages. It does not mean there were 3.05B unique tokens: dataset sources overlap by design and the same dataset revision is used for continued pretraining. The stage shuffle seeds differ, but changing a shuffle seed alone does not establish zero overlap between training examples.

5.4 Tokens per parameter

Tokens per parameter is a simple scale statistic:

$$\text{Tokens per parameter} = \frac{T_{\text{total}}}{P}$$

For $P = 51{,}082{,}752$:

$$\frac{3{,}050{,}037{,}248}{51{,}082{,}752} \approx \boxed{59.71}$$

Point in pipeline	Cumulative processed positions	Tokens per parameter (approx.)
After Stage 1	49,999,872	0.98
After Stage 2	99,999,744	1.96
After Stage 3	1,050,009,600	20.56
After Stage 4	3,050,037,248	59.71

The decision to expand Stage 3 toward one billion cumulative pretraining positions was influenced by compute-optimal training research, including the Chinchilla study [3], often discussed through a rough tokens-per-parameter heuristic. The frequently repeated "about 20 tokens per parameter" is not a universal requirement or guarantee. TinyGPT reached approximately 20.56 by the end of Stage 3, then continued to approximately 59.71 after Stage 4. This ratio alone does not show that the final training budget was compute-optimal or that the model must be capable; data quality, architecture, optimization, objective, training stability, and evaluation all matter. For this parameter count, 3.05B processed positions is about 3.0 times the rough 20-tokens-per-parameter heuristic. Therefore, the final model's weak generation performance should not be attributed only to an insufficient pretraining-token count.

5.5 SFT result deltas

Using the Stage 4 SFT evaluation values:

$$\Delta L_{\text{chat,MixA}} = 3.2886 - 2.6181 = \boxed{0.6705}$$

$$\Delta L_{\text{Q\&A,MixB}} = 3.4051 - 3.1494 = \boxed{0.2557}$$

Mix A has the lowest recorded chat validation loss; Mix B has the lowest recorded Q&A validation loss. Because Mix A and Mix B were trained on different total active-token budgets, these are descriptive results rather than a perfectly controlled causal comparison.

6 Final SFT results and distribution shift

Model variant	Chat validation loss	Chat PPL	Q&A validation loss	Q&A PPL
Stage 4 base	3.2886	26.81	3.4051	30.12
Mix A	2.6181	13.71	3.3462	28.39
Mix B	2.8018	16.47	3.1494	23.32

Best observed loss: Mix A for chat validation; Mix B for Q&A validation.

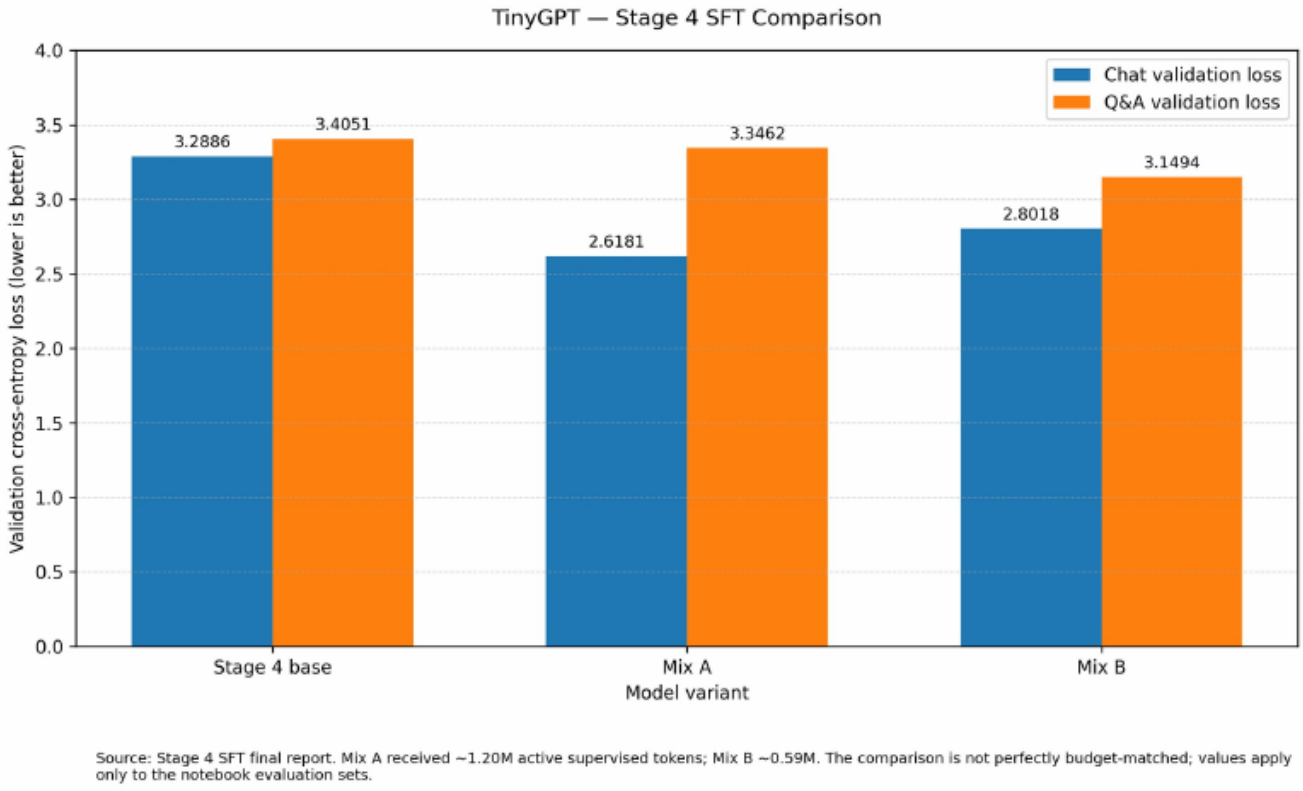
Interpretation: Mix A better predicts the target tokens in the chat validation set; Mix B better predicts the target tokens in the Q&A validation set. This does not mean either variant is uniformly superior or that its answers are always correct.

The SFT notebook also performed a pretraining-distribution drift check using the attached Stage 4 FineWeb-Edu validation cache (the notebook labels this check “FineWeb”):

Model	FineWeb-Edu validation-cache loss in Stage 4 SFT drift check	Perplexity (e^L)
Stage 4 base measured in SFT notebook	3.6558	38.70
Mix A	4.2180	67.90
Mix B	3.9429	51.57

This suggests that adapting to the SFT response distribution came with worse loss on the attached FineWeb-Edu pretraining-distribution cache, particularly for Mix A. The `stage4-sft.ipynb` code explains why its base measurement (3.6558) differs from Stage 4 pretraining’s 3.6472 even though it used the same 977-chunk cache (250,112 input positions): the pretraining evaluator uses batches of 16, records each batch’s mean loss, and averages the 62 batch means equally; because 977 = 61 × 16 + 1, the one-chunk remainder batch receives the same weight as a full 16-chunk batch. The SFT drift-check evaluator instead sums cross-entropy over target tokens and divides by the total number of scored target tokens, yielding a token-weighted mean. These aggregation methods are different, so the figures should not be treated as identical estimates. For each full 256-token chunk, causal shifting yields 255 target positions.

Figure 5. Final SFT comparison



The grouped bars compare the Stage 4 base, Mix A, and Mix B on the held-out chat and Q&A losses. The comparison is not perfectly budget-matched; the values apply only to the notebook evaluation sets.

7 Generation benchmark and qualitative findings

The final playground loads existing checkpoints and does not train them. The isolated benchmark used 27 prompts, four decoding configurations, and two variants (Mix A and Mix B), producing up to 216 model responses. Every test used a freshly tokenized input; the benchmark did not carry over

previous playground history or a prior key-value cache. The decoding configurations included greedy generation and sampled generation, with settings such as temperature, top-p, top-k, repetition penalty, and a 96-token response cap. Prompt categories included identity, conversation, empathy, factual knowledge, everyday knowledge, mathematics, logic, coding, hallucination resistance, honesty, and instruction-following. Repetition flags in the report are heuristics and are not accuracy scores. The benchmark was a small project-specific probe, not an independently standardised evaluation suite.

Representative observations from the saved report:

- Some simple knowledge was handled correctly: both variants answered that the capital of France is Paris.
- Arithmetic failed: for "12 apples, buy 8, give 5 away," both variants answered 12 instead of 15.
- Coding failed: for a prompt requesting a Python square function, the responses described the function rather than providing a working implementation.
- Hallucination resistance was weak: both variants invented an exact room temperature despite having no access to room sensors or location-specific measurements.
- Honesty and premise correction were weak: responses claimed that every answer could be guaranteed correct or failed to appropriately correct false premises.
- Repetition and response relevance remained concerns, in particular on longer answers and some factual/instruction-following prompts.

The four decoding configurations gave the following repeat-diagnostic results in the saved `benchmark_report.txt`:

Configuration	Repetition penalty	Duplicate-response counts (Mix A / Mix B)	Responses flagged for repeated n-grams (Mix A / Mix B)
`greedy_neutral`	1.00	0 / 0	4 / 6
`greedy_repeat_penalty`	1.15	0 / 0	0 / 1
`sample_balanced`	1.10	0 / 0	0 / 0
`sample_conservative`	1.15	0 / 0	0 / 1

The duplicate-response heuristic found no identical normalized response reused across different prompts for any model/configuration. A repetition penalty of 1.15 greatly reduced the repeated-n-gram flags in greedy decoding but did not eliminate the flag entirely for Mix B in all settings. These flags are diagnostic heuristics, not a measure of correctness.

Under `sample_balanced`, Mix A responded to "I had a really difficult day today" with "I am sorry to hear that. What happened?" and followed the programming-concept prompt with "What did you learn?" Mix B produced the less appropriate "Yeah, that's good for you…" response to the difficult-day prompt, although it also asked "What did you learn?" after the programming-concept prompt. This small qualitative example suggests a difference in conversational tone on these prompts, not a general performance guarantee.

No global "accuracy percentage" is reported because the benchmark was not fully scored against a carefully defined answer key, and the heuristic flags do not determine correctness. The measured losses and qualitative prompts address different aspects of the model and should be interpreted together.

8 GGUF conversion and local deployment

The final conversion notebook converts the Stage 4 SFT best checkpoints to GGUF using the llama.cpp Hugging Face-to-GGUF converter. The export type is F16 / FP16 GGUF, not an integer-quantized format. Conversion does not train or fine-tune the model.

Artifact	Source checkpoint	Format	Recorded size
`TinyGPT-v2-Mix-A-F16.gguf`	`sft_mixA_best.pt`	FP16 GGUF	≈99.48 MiB
`TinyGPT-v2-Mix-B-F16.gguf`	`sft_mixB_best.pt`	FP16 GGUF	≈99.48 MiB

The conversion notebook checks the GGUF signature and conversion output. We subsequently imported and tested the exported variants in LM Studio. Successful file conversion and loading should not be confused with strong generation quality: the qualitative benchmark shows substantial limitations. The 256-token context length also places a practical limit on longer prompts and conversation history.

9 Limitations and threats to validity

- **Limited model capacity.** At approximately 51M parameters, TinyGPT is much smaller than contemporary general-purpose assistant models. The model should be treated as an experimental small language model.
- **Short context window.** The model's configured context length is 256 tokens. Long prompts/history must be truncated or omitted, limiting multi-turn use.
- **Processed tokens are not unique tokens.** The cumulative 3.05B count aggregates processed input-token positions across stages and does not imply 3.05B unique, non-overlapping text tokens.
- **Validation comparability.** Stage 1/Stage 2, Stage 3/Stage 4, SFT chat/Q&A, and FineWeb drift-check losses originate from different evaluation sets or procedures. They must not be blindly ranked as a single common metric.
- **Unequal SFT exposure.** Mix A has approximately 1.20M active supervised tokens, versus approximately 0.59M for Mix B. The comparison is not fully controlled for token budget.
- **Small qualitative benchmark.** The 27-prompt benchmark is useful for revealing failure modes but is not statistically comprehensive and is not a recognised external leaderboard evaluation.
- **Known generation failures.** Arithmetic, coding, factual reliability, instruction following, repetition, and hallucination resistance remain weak.
- **Incomplete early history.** The saved Stage 2 SFT notebook output does not preserve a complete reliable epoch-level metric history. No missing metrics were reconstructed.
- **Data provenance and licensing.** Dataset IDs are recorded in the notebooks, but the exact versions, licenses, use restrictions, attribution requirements, and compatibility for redistributing a derived model must be reviewed before publication. Do not assign a permissive model license until this review is complete.
- **Reproducibility depends on source artifacts.** The notebooks reference Colab/Kaggle paths and attached checkpoints/data. A clean clone alone will not reproduce the entire training run unless the correct data, checkpoints, environment versions, and accelerator availability are supplied.

9.1 Why the conversational objective was not met

Several factors may have contributed, but the available experiments do not isolate their individual causal effects. The model is very small (51M parameters), has a short 256-token context, and uses a GPT-2-style Transformer and general-purpose GPT-2 tokenizer rather than a tokenizer or architecture specifically adapted to this project. The final SFT stage used relatively small active-token budgets (approximately 1.20M for Mix A and 0.59M for Mix B), and the project did not include preference tuning or an RLHF/DPO-style alignment stage. These are plausible limitations, not experimentally isolated causes. Importantly, the model processed approximately 3.05B input-token positions, or 59.71 positions per parameter—roughly three times the often-cited ~20 tokens-per-parameter Chinchilla heuristic. Thus, the observed weakness should not be explained solely as "too few pretraining tokens"; capacity, data quality and coverage, post-training, architecture, optimization choices, and evaluation all remain relevant.

9.2 Difference from industry-scale language-model systems

This project follows a recognisable high-level language-model pipeline, but it differs substantially from pipelines commonly used for industry-scale assistants. Such systems generally involve much

larger models and datasets, more extensive source filtering, deduplication and quality/safety curation, broader post-training mixtures, preference optimization or RLHF-style stages, more advanced architectural/training components, and larger teams with substantially greater compute and evaluation resources. Exact proprietary training details vary and are not asserted here. This is a qualitative comparison, not a reconstruction of any particular commercial model's training process.

10 Conclusion

The original objective was to build a good conversational AI model from scratch. TinyGPT did not achieve that objective: the final Mix A and Mix B models remain weak on reasoning, arithmetic, coding, instruction-following, factual grounding, and hallucination resistance. This outcome is consistent with the limitations of a 51M-parameter model and the modest SFT budgets used here, but the study does not isolate any single cause.

The model processed approximately 3.05B input-token positions (about 59.71 positions per parameter), so the observed shortfall cannot be reduced to a simple claim that pretraining tokens were insufficient. The project did succeed in carrying out and documenting the full engineering workflow: random-initialized GPT-2-style model configuration, four-stage pretraining, checkpoint continuation, SFT data preparation, two SFT variants, quantitative loss/perplexity analysis, prompt-based evaluation, and FP16 GGUF deployment. The clearest experimental finding is that improvements in task-specific validation loss—Mix A for chat and Mix B for Q&A—did not translate into consistently reliable generated answers. That distinction, together with the actual failure cases and the documented lineage, is the central result of the project. The outcome is valuable because the process and its limits are visible and auditable, not because the final model is production-ready.

11 Artifacts and reproducibility

The project artifacts comprise this technical report, the eight supplied training/evaluation/conversion notebooks (preserved as experiment records), the consolidated training-history workbook and benchmark report, five figures, and two exported FP16 GGUF model files: `TinyGPT-v2-Mix-A-F16.gguf` and `TinyGPT-v2-Mix-B-F16.gguf`. The GGUF files were imported into and tested with LM Studio, but the binary models are distributed separately and are not included in this documentation bundle. Their download location and model cards should be linked from the project README or model-release page after dataset licensing and redistribution conditions have been reviewed.

The notebooks are retained as provided, including their Colab/Kaggle-specific paths, cached outputs, and historical implementation choices. A fresh clone alone is not sufficient to reproduce the full runs: reproduction also depends on dataset revisions/splits, saved checkpoints, notebook execution order, package/library versions, GPU availability, and external storage paths. Missing Stage 2 SFT epoch metrics were not reconstructed. The benchmark is a small project-specific probe, not a standardized leaderboard evaluation.

AI assistance disclosure: The notebook code was written with AI assistance (ChatGPT) under the team's direction. The team directed the pipeline design, configured and ran the training experiments, reviewed outputs, and carried out the evaluation and deployment workflow. AI assistance is acknowledged as part of the implementation process; it does not replace the team's responsibility for the reported experiments and claims.

References

Technical background

[1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need.* Advances in Neural Information Processing Systems 30. https://arxiv.org/abs/1706.03762

[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI technical report. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[3] Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). *Training Compute-Optimal Large Language Models.* Advances in Neural Information Processing Systems 35. https://arxiv.org/abs/2203.15556

[4] Penedo, G., Kydlíček, H., Ben allal, H., et al. (2024). *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale.* Advances in Neural Information Processing Systems 37, Datasets and

Benchmarks Track. https://arxiv.org/abs/2406.17557

[5] Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization.* International Conference on Learning Representations. https://arxiv.org/abs/1711.05101

Project evidence reviewed for this report

[P1] `stage1-2.ipynb` — initial FineWeb pretraining, corrected Stage 2 continuation, early DailyDialog SFT, and saved outputs.

[P2] `stage-3-pretrain.ipynb` and consolidated Stage 3 training log — Stage 3 configuration, session continuation, token counts, and validation loss history.

[P3] `stage-3-sft.ipynb` and consolidated training log — experimental Stage 3 SFT mixtures and evaluation values.

[P4] `stage-4-pretrain.ipynb` and consolidated Stage 4 training log — Stage 4 configuration, continuation, processed token counts, and validation history.

[P5] `stage-4-sft-data-prep.ipynb` — final SFT data preparation and source counts.

[P6] `stage4-sft.ipynb` — final Mix A / Mix B configurations, checkpoints, validation metrics, and FineWeb drift check.

[P7] `stage-4-final-testing-playground.ipynb` and `benchmark_report.txt` — isolated 27-prompt generation benchmark and recorded responses.

[P8] `final-gguf-conversion.ipynb` — FP16 GGUF conversion configuration and output checks.

[P9] `TinyGPT_v2_Training_History.xlsx` — structured summary and extracted training/validation history supplied with this report.